

Introduction to Computers and Programming in C

Module V

Strings

Strings

- Character arrays are known as strings
- Character arrays or strings are the data types used by programming language to manipulate text such as words and sentences
- A string constant is a one-dimensional array of characters terminated by '\0'(Null character)

Strings

- `Char str1={'A','B','C','D','\0'};`
- `Char str1[]="ABCD";` `//'\0'` will be automatically inserted
- `'\0'` represents the end of the string. Also referred to as string terminator & Null character.

String Input/Output

- `Printf("%s",str1);`
- `Puts(str1);`
- `Scanf("%s",&str1);`
- `Gets(str1);`

String Input/Output

```
int main()
{
    char name[20];
    printf("Enter your name:");
    scanf("%s", name);
    printf("%s",name);
    return 0;
}
```

String Input/Output

```
int main()
{
    char name[20];
    printf("Enter your name:");
    gets(name);
    puts(name);
    return 0;
}
```

C String Function

- Strlen

For C string function we need to include header file `<string.h>`

The C library function **size_t strlen(const char *str)** computes the length of the string but not including the terminating null character

size_t represents unsigned short is the result of the **sizeof** keyword

C String Functions

```
#include <string.h>

int main()
{
    char str1[20] = "Programming in C";
    printf("Length of string str1: %d",
        strlen(str1));
    return 0;
}
```


C String Functions

- `strlen` vs `sizeof`

`strlen` returns you the length of the string stored in array, however `sizeof` returns the total allocated size assigned to the array

- In the example discussed in the last slide `strlen` will return 16 and `sizeof` will return 20

C String functions

- Strnlen

`size_t strnlen(const char *str, size_t maxlen)`

It returns length of the string if it is less than the value specified for maxlen (maximum length) otherwise it returns maxlen value

C String functions

```
int main()
{
    char str1[20] = "Programming in C";
    printf("Length of string str1 when maxlen is
30: %d", strlen(str1, 30));
    printf("Length of string str1 when maxlen is
10: %d", strlen(str1, 10));
    return 0;}
```

C String Functions

- Strcmp()

`int strcmp(const char *str1, const char *str2)`

It **compares two strings** and returns an integer value. If both the strings are same (equal) then this returns 0 otherwise it may return a negative or positive value based on the comparison

C string Functions

```
int main()
{
char s1[20] = "Programming in C";
    char s2[20] = "ANSI C";
    if (strcmp(s1, s2) == 0)
    {
        printf("string 1 and string 2 are equal");
    }
else
    {
        printf("string 1 and 2 are different");
    }
return 0;}
```

C string Functions

- `strncmp()`

```
int strncmp(const char *str1, const char  
*str2, size_t n)
```

It **compares both the string till n characters**
or in other words it compares first n
characters of both the strings.

C String Functions

```
int main()
{
char s1[20] = "Programming in C";
    char s2[20] = "ANSI C";
    if (strncmp(s1, s2,5) ==0)
    {
printf("string 1 and string 2 are equal");
    }
else
{
    printf("string 1 and 2 are different");
}
return 0;}
```

C String Functions

- strcat()

`char *strcat(char *str1, char *str2)`

It concatenates two strings and returns the concatenated string.

C string Functions

```
int main()
{
    char s1[20] = "Programming";
    char s2[10] = " in C";
    strcat(s1,s2);
    printf("Output string after concatenation: %s", s1);
    return 0;
}
```

C string functions

- `strncat()`

`char *strncat(char *str1, char *str2, int n)`

It **concatenates n characters of str2 to string str1**. A terminator char (`'\0'`) will always be appended at the end of the concatenated string.

C string Functions

- strcpy()

`char *strcpy( char *str1, char *str2)`

It copies the string str2 into string str1, including the end character (terminator char `'\0'`).

C string Functions

```
int main()
{
char s1[] = "string 1";
char s2[] = "string 2 : I'm gonna copied into s1";
/* this function has copied s2 into s1*/ strcpy(s1,s2);
printf("String s1 is: %s", s1);
return 0;
}
```

C string Functions

- `strncpy()`

`char *strncpy( char *str1, char *str2, size_t n)`

`size_t` is unsigned short and `n` is a number.

Case1: If length of `str2` $> n$ then it just copies first `n` characters of `str2` into `str1`.

Case2: If length of `str2` $< n$ then it copies all the characters of `str2` into `str1` and appends several terminator chars(`'\0'`) to accumulate the length of `str1` to make it `n`

References

- Programming in ANSI C, E. Balagurusamy. McGrawHill
- Let Us C, Yashvant Kanetkar, BPB Publications
- Programming in C, Reema Thareja, Oxford University Press

Thank You